

HW4: Model Validation Report

一：代码运行

Price 模型运行时出现的错误

1、KMeans 内存泄漏警告（我的电脑问题，代码本身没问题）

```
KMeans地理聚类处理开始...
按板块聚合后，得到 973 个板块的中心经纬度
E:\python\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less
chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=4.
warnings.warn()
```

这是一个在 Windows 系统上运行 KMeans 聚类时常见的警告信息，属于 scikit-learn 库与 Intel MKL 数学库在特定环境下的兼容性问题，严格来说并不严重。它只是一个警告而非错误，意味着你的聚类分析会继续正常执行并产生正确的结果，主要是在内存管理方面存在潜在的小问题，但不会影响 973 个板块地理聚类的计算准确性。

2、特征工程——删除共线性较高的特征这步出现错误（代码问题）

```
File E:\python\Lib\site-packages\statsmodels\base\data.py:675, in handle_data(endog, exog, missing, hasconst, **kwargs)
    672     exog = np.asarray(exog)
    674     klass = handle_data_class_factory(endog, exog)
--> 675     return klass(endog, exog=exog, missing=missing, hasconst=hasconst,
    676                  **kwargs)

File E:\python\Lib\site-packages\statsmodels\base\data.py:88, in ModelData.__init__(self, endog, exog, missing, hasconst, **kwargs)
    86     self.const_idx = None
    87     self.k_constant = 0
--> 88     self._handle_constant(hasconst)
    89     self._check_integrity()
    90     self._cache = {}

File E:\python\Lib\site-packages\statsmodels\base\data.py:134, in ModelData._handle_constant(self, hasconst)
    132     exog_max = np.max(self.exog, axis=0)
    133     if not np.isfinite(exog_max).all():
--> 134         raise MissingDataError('exog contains inf or nans')
    135     exog_min = np.min(self.exog, axis=0)
    136     const_idx = np.where(exog_max == exog_min)[0].squeeze()

MissingDataError: exog contains inf or nans
```

这是一个比较严重的数据库质量问题，statsmodels 库在检查你的自变量数据时发现了无穷大值 (inf) 或缺失值 (NaN)，导致模型无法进行正常的数学计算而报错终止。虽然不会损坏你的原始数据，但必须彻底清理数据中的这些异常值后才能继续进行分析建模，否则任何统计模型都无法正常运行。

```
# 数据诊断 - 找出具体的Inf值位置
def diagnose_inf_values(df):
    """诊断数据中的Inf值"""
    numeric_df = df.select_dtypes(include=['int64', 'float64'])

    print("=== Inf值诊断报告 ===")
    inf_columns = []

    for col in numeric_df.columns:
        col_inf = np.isinf(numeric_df[col]).sum()
        if col_inf > 0:
            inf_columns.append(col)
            print(f"列 '{col}': {col_inf} 个Inf值")

            # 显示一些样本值
            sample_inf = numeric_df[numeric_df[col].isin([np.inf, -np.inf])][col].head(3)
            print(f"样本值: {sample_inf.values}")

    if not inf_columns:
        print("✅ 未发现Inf值")
    else:
        print(f"\n总计: {len(inf_columns)} 个列包含Inf值")

    return inf_columns

# 运行诊断
inf_columns = diagnose_inf_values(X_train_final)

=== Inf值诊断报告 ===
✅ 未发现Inf值
```

诊断发现无 inf 值

=== 全面数据质量诊断 ===

✗ 发现 80604 个NaN值

列 '绿化率': 80604 个NaN值

✗ 发现常量列: ['绿化率']

📊 数据形状: (80604, 31)

📊 数据类型分布:

float64 21

int64 10

Name: count, dtype: int64

诊断发现，“绿化率”列存在空缺值，需要将这一列移除或者其他处理消灭这些空缺值，特征工程处理以及建模才能正常运行

3、处理区域特征，进行独热编码出现报错

```
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
C:\Users\杨\AppData\Local\Temp\ipykernel_28384\1724111828.py:36: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling 'frame.insert' many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use 'newframe = frame.copy()'
test_encoded[col] = 0 # 测试集无此区域类别, 填0
```

这是一个完全不严重的性能警告，只是提示你的 DataFrame 在内存中变得碎片化从而略微影响运行效率，但完全不会影响代码的正确性和最终计算结果，你完全可以忽略这个警告继续进行分析工作。

Rent 模型运行时出现的错误

无

二：数据处理

该机器学习项目的数据处理流程整体设计规范、逻辑连贯，全面覆盖了数据预处理的基本环节，充分保留了原始数据集的有效信息，并针对性解决了数据异构、缺失、异常等关键问题，为后续模型训练奠定了坚实基础。其对 price 和 rent 数据的处理流程由于数据集特征列类型、形式不同而略有差异，但基本逻辑如下：

1.数值转换

针对含有数值的数据列进行数值转换，将包含单位的字符串型数据进行处理，得到浮点型、整型数据，同时对区间值进行处理，得到仅包含浮点数、整数、分类变量、二值变量与缺失值的数据集。

该步骤解决了数据类型异构的核心问题，通过标准化处理将非规范格式转化为可用于模型训练的标准化数据类型，彻底消除了格式差异对后续分析的干扰，使后续样本筛选、缺失值填补等环节得以在统一且规范的数据集上进行，处理思路贴合实际数据特点，具有一定实用性。

2.样本筛选与特征初步筛选

删除训练集中缺失值过多的样本（缺失 28 个及以上特征的样本），同时删除无效或者高度缺失的特征列，避免影响后续缺失值填补和模型训练过程。

该步骤具有明确目标导向性，通过剔除高缺失样本和无效、高缺失特征，显著减少了数据噪声与冗余信息，降低了后续缺失值填补的偏差风险，并精简了特征维度，是提升数据质量与模型训练效率的合理步骤。

3.缺失值填充

针对不同类型特征列的性质和逐一、按列处理缺失值，其思路主要为对连续变量缺失值采用中位数填充，对分类变量缺失值则用众数填充（另有对部分分类变量特征列采用“未知”填充缺失值），最后将分类特征转化为数值型，进行独热编码。

整体填充逻辑符合数据预处理的常规原则，利用中位数对连续变量的极端值不敏感、众数能反映分类变量的主流分的性质，兼顾了数据特性与合理性。后续的独热编码进一步完成了分类特征的数值化转换，适配了机器学习模型的数值输入需求。但需重点关注的是，忽略区县、板块的异质性采用全局统一填充，可能会掩盖区域特异性带来的数据分布差异，进而导致填充值与真实数据分布偏离，影响后续模型的预测准确性与泛化能力。

4.文字描述特征数据处理

按特征列分别整理文字描述关键词词典，按识别频次记分转化为整型变量。

通过关键词频次记分实现非结构化文字数据的量化，思路简洁高效且可操作性强，将无固定格式的文字信息转化为结构化数值特征，为模型充分该类信息提供了可行路径。但文字长度对频次的干扰问题有待进一步解决。由于长文本天然包含更多词汇，其关键词频次可能高于短文本，导致频次得分无法真实反映特征本质；若补充文字长度标准化（如计算关键词频次占文本总词数的比例），可有效抵消描述长度对词汇频次造成的干扰，进一步提升特征有效性。

5.异常值处理

利用 IQR 法对异常值进行筛选，将异常值替换为边界值。

采用 IQR 法处理异常值作为一种稳健的方案，相较于直接删除异常值的方式，能够避免极端值对模型参数估计的干扰，且最大程度保留了数据的原始分布与样本完整性，减少有效数据的浪费，处理方式科学合理。

整体总结

该数据处理流程围绕“非结构化数据量化→噪声样本行与特征列剔除→缺失值填补→文本信息处理→异常值修正”的逻辑展开，覆盖了预处理全流程，方法选择贴合数据类型与实际需求，充分保留了原始数据的有效信息，为后续模型训练提供了高质量的输入数据。同时，若后续针对缺失值填补时未考虑区域异质性、文字描述类特征信息未经标准化等细节进一步优化，或能进一步提升数据质量，让模型更充分地学习到数据中的关键规律，进而优化预测效果。

三：样本选择

从代码实现来看，该项目的样本选择和数据处理流程存在一些潜在的数据泄露风险和样本划分问题。首先，在数据预处理阶段，所有特征工程操作（包括缺失值填充、特征编码、标准化等）都是在整个训练集上进行的，然后将相同的变换应用于验证集和测试集。这种做法会导致验证集的信息泄露到训练过程中，因为验证集参与了训练阶段的参数计算（如均值、中位数、众数等）。具体表现在：缺失值填充使用了整个训练集的中位数和众数，特征编码映射基于整个训练集构建，标准化参数基于整个训练集计算。这种数据泄露会使模型在验证集上的表现过于乐观，无法真实反映模型在完全未知数据上的泛化能力。

其次，样本划分虽然采用了常规的 8:2 比例，但划分时机存在问题。应该在数据预处理之前就进行样本划分，确保训练集、验证集和测试集完全独立。当前的流程是先在完整训练集上进行特征工程，然后划分验证集，这违背了机器学习的基本原则。另外，对于时间序列特性的租金数据，简单的随机划分可能不是最优选择，因为租金数据通常具有时间依赖性，相邻时间点的样本可能相关。

在异常值处理方面，使用基于整个训练集的统计量（如 IQR 方法）来识别和删除异常值，也会引入数据泄露问题。正确的做法应该是仅基于训练子集计算异常值阈值，然后应用于验证集。

四：建议

建议一：改进缺失值处理策略

分析：当前缺失值处理存在硬编码的填充值分散在多个地方，难以维护和调整。使用随机填充区县和板块的方法可能引入不合理的噪声，影响模型稳定性。填充策略缺乏对特征重要性的考虑，所有数值特征统一使用中位数填充，没有根据特征与目标变量的相关性进行差异化处理。测试集处理逻辑与训练集耦合，容易产生数据泄露问题。整体缺失值处理流程缺乏统一的管理机制，导致代码重复且难以维护。

具体操作建议：创建统一的缺失值处理类，根据特征类型、缺失比例以及与目标变量的相关性自动选择最优填充策略。对于分类变量，根据缺失率选择使用众数或"未知"类别填充。对于数值变量，基于与目标变量的相关性选择中位数或零值填充，重要特征使用中位数，不重要特征使用零值。避免使用随机填充方法，改用频率最高的值进行填充。建立统一的配置管理机制，便于后续维护和策略调整。

建议二：优化代码结构和消除冗余操作

分析：代码中存在大量重复的模式处理逻辑，特别是在特征编码部分，每个特征的处理都是独立的代码块，导致代码冗长且维护困难。缺乏统一的特征处理接口，函数职责不单一，耦合度高。在特征选择过程中，存在多次删除操作和多次数据对齐，导致代码冗余且执行效率低下。测试集处理逻辑分散在多处，缺乏统一的特征管理策略，容易产生数据泄露问题。硬编码的列名和特征选择标准分散在各处，使得代码难以理解和修改。

具体操作建议：创建统一的特征处理器类，通过配置驱动的方式处理不同类型特征，将每个特征的处理逻辑封装成配置项，提高代码复用性和可维护性。使用统一的接口处理训练集和测试集，确保处理逻辑的一致性。合并所有要删除的特征，一次性处理，减少冗余操作。创建统一的特征管理器类，基于统计指标自动选择特征，实现特征重要性评估和报告功能，便于决策和优化。

建议三：改进文本特征处理方法

分析：当前文本特征处理使用简单的关键词计数方法，忽略了上下文信息和词汇权重差异，没有充分利用文本的语义信息。所有关键词权重相同，无法区分重要设施和普通设施的价值差异。缺乏对文本长度和复杂度的考虑，无法准确评估描述信息的信息量。情感分析仅基于简单计数，没有考虑情感强度和文本语境。整体文本特征处理过于简单，无法捕捉文本中蕴含的丰富信息。

具体操作建议：实现加权关键词计数方法，为稀有设施和强情感词分配更高权重，准确反映不同词汇的重要性差异。添加文本长度和复杂度作为额外特征，评估描述信息的信息量。实现细粒度的情感强度得分计算，考虑词频和语境因素。对设施关键词按稀缺性分配不同权重，稀有设施权重更高，常见设施权重较低。添加文本存在性标识，区分有详细描述和无描述样本，提高特征的表征能力。

建议四：创建有业务意义的交叉特征

分析：当前特征工程缺乏对特征间交互作用的有效捕捉，线性模型无法自动学习这些复杂关系。没有创建能够反映业务逻辑的复合特征，导致模型表达能力受限。缺乏价格密度、空间效率等重要业务指标的构建，无法准确反映房产的实际价值。地理位置与物业类型的交互效应未被建模，忽略了区位对不同类型的房产价值的差异化影响。整体特征工程缺乏对业务理解的深入应用，特征组合的潜力未被充分挖掘。

具体操作建议：创建价格密度特征，通过面积与价格的比值反映单位面积价值。构建空间效率指标，评估户型设计与面积的匹配程度。设计设施完备度与建筑质量的交互特征，反映现代化设施与房龄的综合影响。建立地理位置与物业类型的交互特征，捕捉不同区位对各类房产价值的差异化影响。实现复合质量评分系统，综合装修水平、设施完备度和建筑结构等多个质量相关特征。添加距离衰减效应相关的交互特征，量化地理位置对价值的实际影响。